Week 10: Approximate tensor contraction for the Ising model

Jin-Guo Liu, Xi-Wei Pan

Abstract: Can we locate the Ising transition without sampling spin configurations? We will find it through the magnet's response to heating. Week 9 expressed a partition function as a sum of products of local weights. Here we use compression to carry that calculation to large lattices. We first learn what an SVD retains and discards. A boundary matrix product state then compresses summed rows into a chain of small tensors. Tensor renormalization group instead combines local tensors to shrink the lattice. These methods reveal a specific-heat peak that grows and moves as the magnet grows. Open and periodic lattices will lead us towards the same critical temperature.

Computing a magnet without sampling

1 FEEL

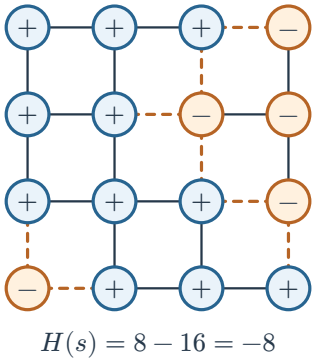
Why are researchers interested in computing the partition function?

Researchers want to predict a material's average energy and how it changes when heated. The partition function connects these measurable quantities to the interactions between particles. We will see this connection in the square-lattice Ising model.

Consider a square lattice whose spins s_i take values $+1$ and -1. Neighbouring spins have energy $-s_i s_j$ in units with coupling $J = 1$. There is no applied field. The total energy is

$$H(s) = -\sum_{(i,j)\in E} s_i s_j. \tag{1}$$

Here E is the list of bonds.



$H(s) = 8 - 16 = -8$

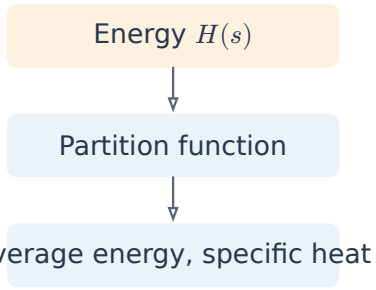


Figure 1: Left: one configuration of a 4×4 magnet. Solid bonds join aligned spins and contribute -1; dashed bonds join opposed spins and contribute $+1$. Right: the partition function turns the energies of all configurations into measurable thermal properties.

At temperature T, lower-energy configurations have larger statistical weights. The Boltzmann weight is $w(s) = \exp(-KH(s))$, with $K = \frac{1}{T}$ and Boltzmann's constant set to one. To turn these relative weights into probabilities, divide by their total. That total is the partition function:

$$Z(K) = \sum_s w(s), \quad p(s) = \frac{w(s)}{Z(K)}. \tag{2}$$

Its dependence on temperature also tells us the average energy:

$$\langle H \rangle = -\frac{d \log Z}{dK}. \tag{3}$$

Here $\langle H \rangle$ means the average over configurations with probabilities $p(s)$.

How does spin alignment change as we heat the square-lattice Ising model?

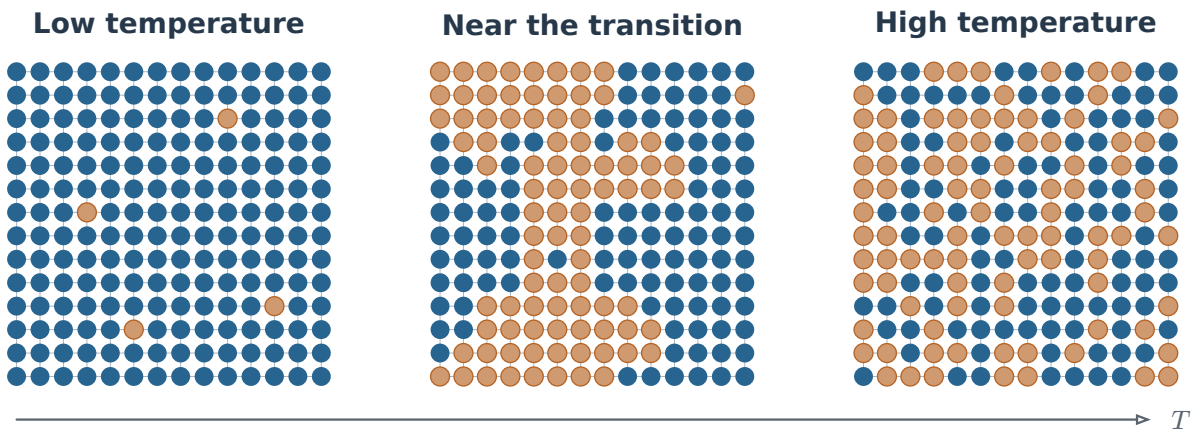


Figure 2: Schematic configurations, not simulation samples; blue marks $s_i = +1$ and orange $s_i = -1$. Heating takes the magnet from near-complete alignment, through domains of many sizes, to no long-range order. Either overall spin sign is possible.

At low temperature, spins tend to align across the magnet. At high temperature, this long-range alignment disappears. The infinite square lattice has a sharp transition between these regimes. Near the transition, spins fluctuate together over long distances. Preserving these correlations can require keeping more information in our compressed tensors.

Our target is the temperature at which the magnet changes its collective behaviour. The partition function gives us a route to it. We measure $g = \frac{\log Z}{N}$, where $N = L^2$ is the number of sites. The *free energy* per site is $f = -Tg = -\frac{g}{K}$ for $K > 0$. We report g so that the infinite-temperature limit $K = 0$ remains finite. At that limit, all configurations have equal weight and $g = \log 2$.

How can we locate the transition temperature from the partition function?

The specific heat per spin measures the energy response to heating:

$$c_V = \frac{1}{N}\frac{d\langle H\rangle}{dT} = K^2\frac{d^2 g}{dK^2}. \tag{4}$$

For each lattice size L, compute $g(K)$ over a temperature range. Its curvature gives the specific-heat curve. A finite lattice has a rounded peak, not a sharp transition. Call the temperature of its maximum $T_{\text{peak}}(L)$. Extrapolate these peak temperatures to $L \to \infty$ to estimate the critical temperature T_c.

Part 2 follows these peaks on open squares up to 64×64. Part 3 uses coarse-graining to reach a periodic square with over a million spins.

The difficulty is evaluating the partition function for a large magnet.

How can we write the 2D Ising partition function as a tensor network of Boltzmann weights?

Recall Week 9: a tensor contraction sums products of local factors. The Ising energy is a sum over bonds, so its Boltzmann weight is a product:

$$Z(K) = \sum_{s}\prod_{(i,j)\in E}\exp\bigl(Ks_i s_j\bigr). \tag{5}$$

Each bond contributes a two-index tensor $F(s,t) = \exp(Kst)$. The spin indices are shared between neighbouring factors.

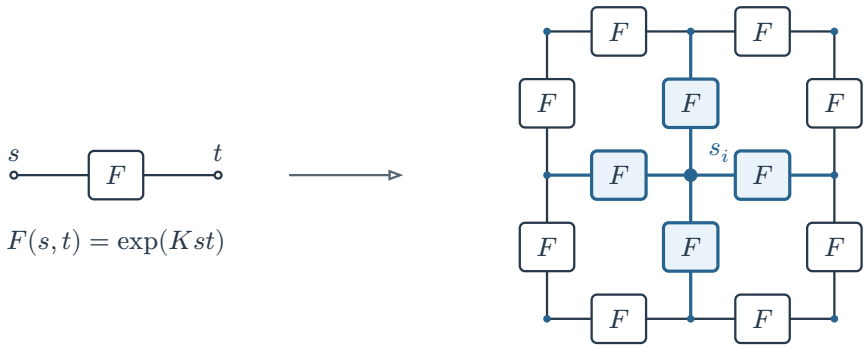


Figure 3: Left: one bond weight F with spin indices s and t. Right: bond weights on a 3×3 patch. Each dot is a spin index shared by the factors around it; the highlighted index s_i is shared by four. Summing every shared index once gives Z.

Each dot joins all the factors that use the same spin index. Summing all shared indices gives the partition function. We can perform these sums one row at a time.

> What is the cost of contracting this 2D network exactly?

The circled region below contains the upper-row spins and the bond factors attached to them.

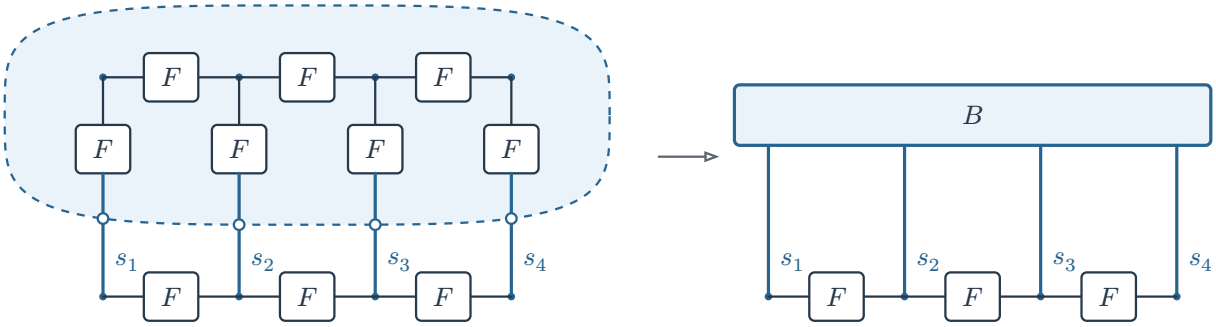


Figure 4: Left: the dashed region holds the upper-row spins and their bond weights; four index wires cross its boundary (open circles). Right: contracting the region leaves one array $B(s_1, s_2, s_3, s_4)$ with $2^4 = 16$ entries.

Sum the spin indices inside the circled region, keeping the four crossing indices unsummed. Call the resulting array $B(s_1, s_2, s_3, s_4)$. It replaces everything inside the circle without changing the rest of the contraction. We call B a **boundary tensor** because its indices connect the contracted region to the remaining network. Each crossing index has two values, so this array has $2^4 = 16$ entries. For a region spanning L columns, the same construction leaves L binary indices and 2^L entries. An explicit array therefore needs storage that grows exponentially with the lattice width. Updating this array also requires work on exponentially many entries. Row-by-row contraction avoids listing all 2^{L^2} spin configurations, but remains costly for large L. Our next step is to store an approximation of this array. We must then judge how much error that approximation introduces.

Boundaries. An open square has no bonds across its edges. A periodic square joins opposite edges, making a torus. These are different finite systems. Each run must state which one it computes.

Part 1: Compress a matrix

Goal: use spectrum and error plots to decide whether low-rank factors can approximate a Gaussian matrix to 10% relative error with fewer stored entries.

(2) IMAGINE

> How can SVD reduce the memory needed to store a matrix or tensor?

We want to replace the large boundary array with smaller factors. We first learn how to approximate a matrix this way. The SVD, or singular-value decomposition, writes a real $m \times n$ matrix as

$$M = U \operatorname{diag}(s_1, \ldots, s_q) V^T, \quad s_1 \geq s_2 \geq \ldots \geq s_q \geq 0. \tag{6}$$

Here $q = \min(m, n)$. The matrices U and V have sizes $m \times q$ and $n \times q$. In the diagram, i and j label the matrix rows and columns. The vector s contains the singular values; their ordered list is the singular spectrum. Here $\operatorname{diag}(s)$ places these values on a diagonal, and V^T transposes V. The junction shares one index α among U, s and V, as in Week 9.

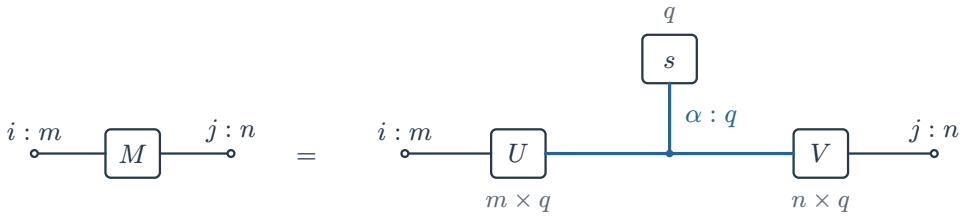


Figure 5: SVD as a tensor network: $M_{ij} = \sum_\alpha U_{i\alpha} s_\alpha V_{j\alpha}$. The shapes of U, s and V are shown beneath them; the single shared index α is summed once.

The columns of U and V are orthonormal directions: perpendicular and of unit length. A singular value s_α measures how strongly the matrix transmits one direction. Keeping the first χ directions gives a matrix M_χ of rank at most χ. Here rank counts independent directions, not tensor legs.

Truncated SVD routine. The algorithms use the thin SVD just defined. Indices start at one; $1 : k$ includes k, and $:$ selects a whole axis. Array assignments read the right-hand entries before replacing the left-hand array.

Algorithm 1. TruncatedSVD(M, χ)

Input: Matrix M and rank cap $\chi \geq 1$.
Output: Factors U, s, V^T retaining at most χ singular directions.

$(U, s, V^T) \leftarrow \mathrm{SVD}(M)$
▷ Keep the strongest directions; discard only the tail.
$k \leftarrow \min(\chi, \mathrm{length}(s))$
return $U[:, 1:k]$, $s[1:k]$, $(V^T)[1:k, :]$.

How many numbers do we need to store after truncating an SVD?

Let U_χ and V_χ contain the first χ columns of U and V. Absorb the retained singular values into the first factor:

$$M_\chi = AB, \quad A = U_\chi \operatorname{diag}(s_1, \ldots, s_\chi), \quad B = V_\chi^T. \tag{7}$$

The factors have sizes $m \times \chi$ and $\chi \times n$.

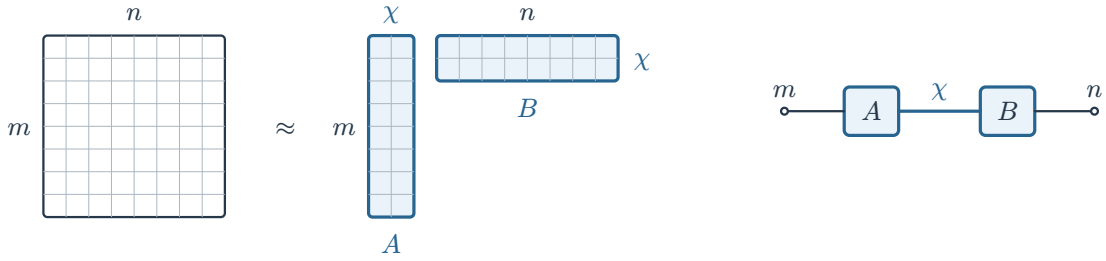


Figure 6: Storage drawn to scale for $m = n = 8$ and $\chi = 2$, one cell per stored entry. The full matrix holds $mn = 64$ entries; the factors $A = U_\chi \operatorname{diag}(s_1, \ldots, s_\chi)$ and $B = V_\chi^T$ hold $m\chi + \chi n = 32$. Right: the same factorization as a tensor network.

We can store these factors instead of the full matrix:

$$mn \quad \text{numbers become} \quad m\chi + \chi n = (m + n)\chi. \tag{8}$$

This counts stored entries, not independent degrees of freedom. For a 1000×1000 matrix with $\chi = 10$, the count falls from $1,000,000$ to $20,000$: 50 times less storage. The saving requires $(m + n)\chi < mn$. An SVD with every direction retained is a factorization, but does not save storage.

If two matrices have low rank, can we multiply them without expanding their factors?

Consider two $n \times n$ matrices already stored in rank-χ form:

$$M_1 = AB, \quad M_2 = CD. \tag{9}$$

Here A, C have size $n \times \chi$, while B, D have size $\chi \times n$. Regroup the multiplication so that the small inner matrix is formed first:

$$M_1 M_2 = A(BC)D = ED, \quad E = A(BC). \tag{10}$$

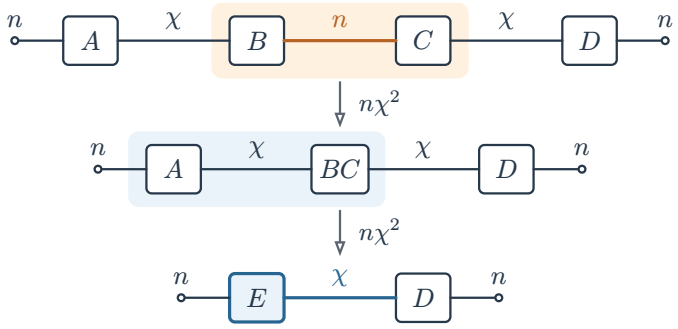


Figure 7: Multiplying two rank-χ matrices in factored form. Each arrow contracts the shaded pair and costs $n\chi^2$ multiplications. The result stays factored as ED with $E = A(BC)$.

The product BC has size $\chi \times \chi$. Computing it uses $n\chi^2$ scalar multiplications. Forming E uses another $n\chi^2$. The result stays in factored form as ED. Ordinary dense multiplication instead uses n^3 scalar multiplications.

For $n = 1000$ and $\chi = 10$, the count falls from 10^9 to 2×10^5. That is 5000 times fewer multiplications, not a measured runtime speedup. These counts exclude the initial SVD. Expanding ED into a full matrix would add $n^2\chi$ multiplications and require n^2 stored entries. Keeping the factors is what preserves the larger saving.

> How much error do we introduce by keeping only χ directions?

The squared Frobenius norm is the sum of the squares of all matrix entries. Truncated SVD minimizes the reconstruction error in this norm among matrices of rank at most χ:

$$\|M - M_\chi\|_F^2 = \sum_{\alpha > \chi} s_\alpha^2. \tag{11}$$

For $M = \begin{pmatrix} 3 & 1 & 0 \\ 1 & 3 & 0 \\ 0 & 0 & 1 \end{pmatrix}$, the singular values are $4, 2, 1$. A rank-one approximation loses squared weight $2^2 + 1^2 = 5$. A rank-two approximation loses only 1. We can now choose how many directions to keep for a desired matrix error.

> Does a small retained rank always give a useful approximation?

A slowly decaying spectrum can leave substantial squared weight in the discarded directions. The hands-on example draws each matrix entry independently from a standard-normal Gaussian distribution. This distribution has mean zero and variance one. We judge accuracy using the relative Frobenius error:

$$e_\chi = \frac{\|M - M_\chi\|_F}{\|M\|_F}. \tag{12}$$

Compare this error with the factor-storage ratio $(m+n)\frac{\chi}{mn}$. Compression is useful only when the accuracy we need is reached with fewer stored entries.

> Our lattice uses tensors with several indices. How can matrix compression help?

Label the four indices of a tensor T as i, j, k, l. Group (i, j) into a matrix row index and (k, l) into a column index. After truncated SVD, reshape the factors into three-index tensors A and B.

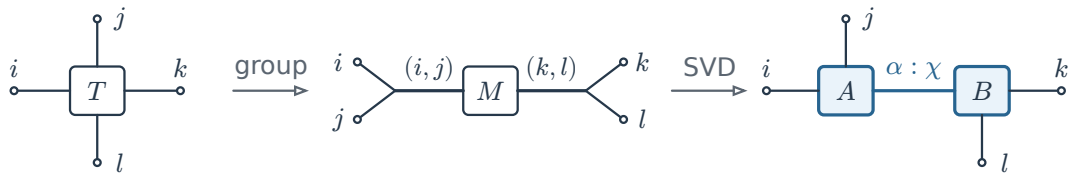


Figure 8: Splitting a four-index tensor. Grouping (i, j) and (k, l) only reshapes T into a matrix M, losing nothing. Truncated SVD then keeps χ singular values: $T_{ijkl} \approx \sum_{\alpha=1}^{\chi} A_{ij\alpha} B_{\alpha kl}$.

Grouping indices changes the shape of the array, not its entries. Information is discarded only when singular values are dropped. The retained singular-value index α becomes the internal leg joining A and B. Its size χ is called the *bond dimension*. Splitting s_α as $\sqrt{s_\alpha}\sqrt{s_\alpha}$ distributes the scale between the two tensors.

Tensor-splitting routine. Ordered groups I, J contain every leg of T once. For the picture, $I = (i, j)$ and $J = (k, l)$. Flatten each group with the last index varying fastest.

Algorithm 2. Split(T, I, J, χ)

Input: Tensor T, ordered leg groups I, J, rank cap χ.
Output: Factors X_{Iq} and Y_{qJ} joined by a retained leg q.

1: $M \leftarrow$ Permute T into order (I, J), then flatten each group into one matrix index.
2: $(U, s, V^T) \leftarrow \mathrm{TruncatedSVD}(M, \chi)$
3: ▷ Share each singular value equally between the two factors.
4: $X_{Iq} \leftarrow U_{Iq}\sqrt{s_q}$
5: $Y_{qJ} \leftarrow \sqrt{s_q}(V^T)_{qJ}$
6: **return** X, Y with I, J unpacked into their original legs.

Gauge freedom on the new bond. We can change the internal basis without changing the represented array. Choose an invertible $\chi \times \chi$ matrix G. Multiply A by G on the shared index and B by its inverse G^{-1}. The inserted matrices cancel when the bond is contracted. This change of basis is a *gauge transformation*.

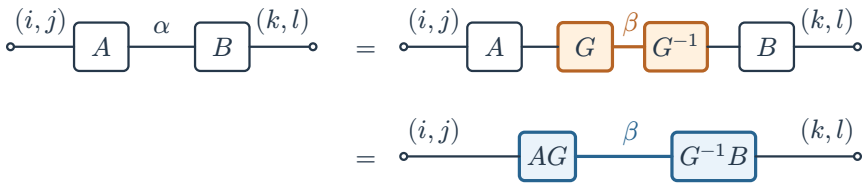


Figure 9: A gauge transformation. Inserting $GG^{-1} = I$ on the bond and absorbing each factor into a neighbour gives the same array in a new bond basis β: $AB = (AG)(G^{-1}B)$.

The external pairs (i, j) and (k, l) are grouped in this picture. A gauge transformation preserves the current approximation; dropping singular values changes it. We will use this freedom to prepare tensor networks for compression.

KEY CONCEPTS

- SVD orders matrix directions by their singular values; truncation keeps the strongest contributions.
- Thin factors reduce stored entries and multiplication work when the retained rank is small.
- The discarded squared singular values give the local reconstruction error; a broad spectrum can prevent useful compression.
- Grouping legs lets the same matrix factorization compress a tensor.
- Gauge transformations change an internal basis without changing the represented array.

③ DO

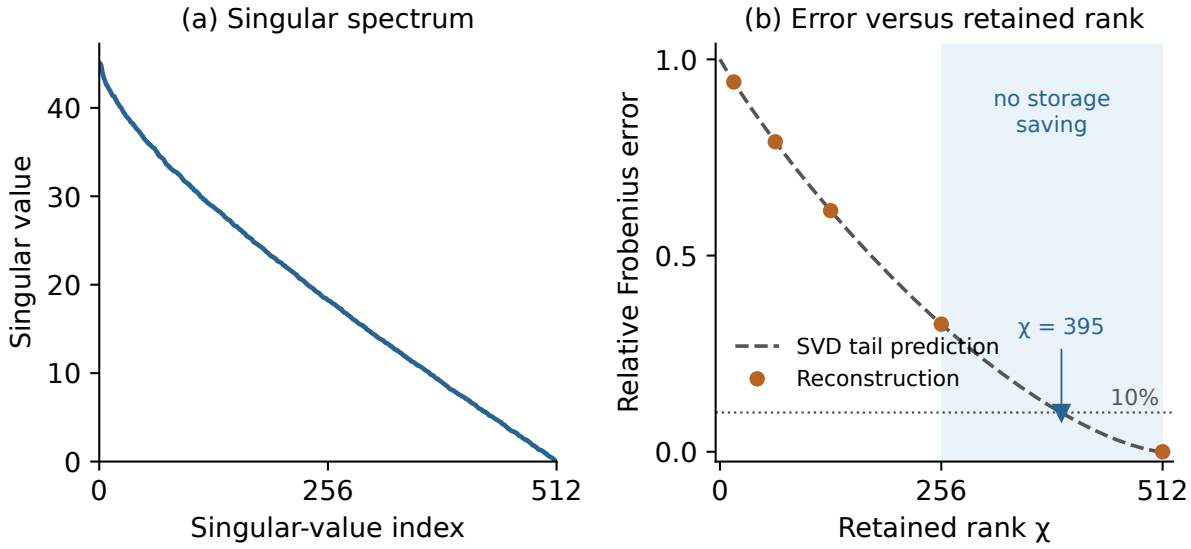
Hands-on: SVD evidence. Work in `week10/` of your own exercise repository. Start one agent conversation and keep it for the following builds.

```
After each build prompt, commit the work before replying. When a scientific
requirement is uncertain, ask me instead of guessing.
```

```
In `week10/`, use Rust to generate a 512×512 matrix with independent standard-normal
Gaussian entries, using seed 2026. Record the random generator. Save the matrix, its
singular values and truncated-SVD reconstructions at ranks 16, 64, 128, 256 and 512.
```

```
Independently recompute relative Frobenius errors from the saved matrices. Plot the
singular spectrum and error versus rank in `artifacts/svd.png`. Overlay the discarded-
spectrum prediction for every rank. Mark where the two-factor storage equals the
original storage and the 10% error threshold.
```

Compression depends on the spectrum. Inspect `artifacts/svd.png`. Each measured relative error must agree with the prediction from your saved singular values within 10^{-8}. At full rank, require $e_{512} < 10^{-8}$. The Rust reference has $e_{16} \approx 0.943$ and $e_{256} \approx 0.326$. Your exact values can differ with the random generator. At $\chi = 256$, the two factors already store as many entries as the original matrix. The error is still above 10%, so smaller ranks cannot meet that accuracy target. The reference first reaches 10% at $\chi = 395$, using about 1.54 times the original storage. Read your crossing from the plot: does this matrix admit a storage-saving approximation at 10% error? All reference plots in this sheet come from an independent NumPy baseline. Printed solver expectations quote the CPU Rust answer key. Each random-matrix result is checked against its own saved spectrum.



Part 2: Contract rows with a boundary MPS

Goal: locate the Ising critical temperature from the growth and drift of specific-heat peaks on open squares up to 64×64.

② IMAGINE

> Why does the specific heat peak near the transition?

Near the transition, many spins rearrange together. The energy then fluctuates more strongly. The specific heat from the introduction also measures these fluctuations:

$$c_V = \frac{K^2}{N}(\langle H^2 \rangle - \langle H \rangle^2). \tag{13}$$

Here $\langle H^2 \rangle - \langle H \rangle^2$ is the variance of the energy. The curvature formula in the introduction gives this response without generating spin samples.

> How can we remove the hyperedges from the Ising tensor network?

A hyperedge is a spin index shared by more than two factors. The bond-gauge cancellation needs two factors. On a hyperedge, a general basis change also changes the shared-index equality constraint. We remove hyperedges to choose a basis on each bond.

Split each bond matrix F into two factors W. Merge four factors around a spin into one site tensor A.

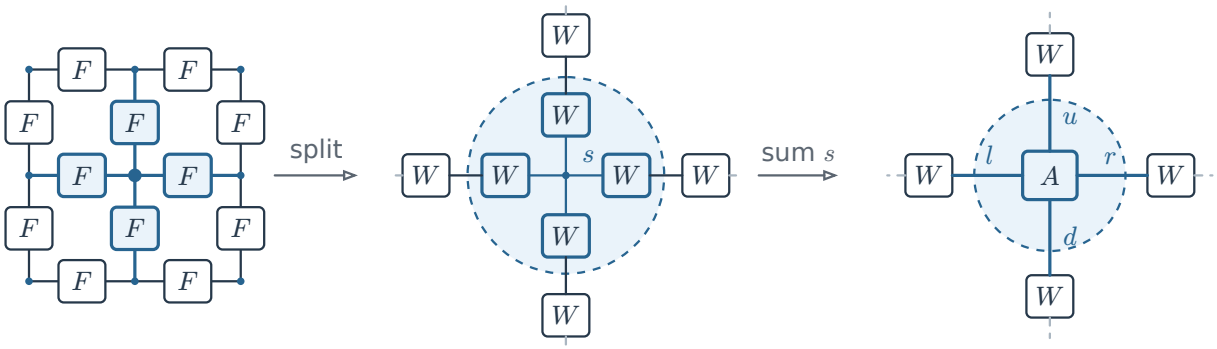


Figure 10: From bond weights to site tensors. Left: a 3×3 patch with the central spin highlighted. Middle: each bond becomes $F = WW^T$; the shaded region holds the four W factors that share spin s. Right: summing s leaves the site tensor A with legs r, u, l, d, each of dimension two.

Square-root splitting of a bond. For $K \geq 0$, the bond matrix has the SVD

$$F = \begin{pmatrix} e^K & e^{-K} \\ e^{-K} & e^K \end{pmatrix} = Q \ \mathrm{diag}(2\cosh K, 2\sinh K) Q^T, \quad Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \tag{14}$$

Share the square root of each singular value between the two endpoints:

$$W = Q \ \mathrm{diag}\left(\sqrt{2\cosh K}, \sqrt{2\sinh K}\right) = \begin{pmatrix} \sqrt{\cosh K} & \sqrt{\sinh K} \\ \sqrt{\cosh K} & -\sqrt{\sinh K} \end{pmatrix}. \tag{15}$$

The square roots act on singular values, not on individual entries of F. The rows of W correspond to $s = +1, -1$. Joining the two halves restores the bond:

$$F = WW^T, \quad \sum_a W_{sa} W_{ta} = \exp(Kst). \tag{16}$$

No singular values have been discarded.

Merging at a site. The blue region in the middle picture contains four W factors and their shared spin. Sum that spin to form

$$A_{ruld} = \sum_{s=\pm 1} W_{sr} W_{su} W_{sl} W_{sd}. \tag{17}$$

The ordered legs are right, up, left and down. Each leg initially has dimension two. The outer W factors stay attached to neighbouring spins. Joining neighbouring legs restores the bond weights. Contracting every index therefore gives Z. For an absent bond at an open edge, attach

$$b = \left(\frac{1}{\sqrt{\cosh K}}, 0\right)^T. \tag{18}$$

Since $Wb = (1, 1)^T$, the absent bond contributes weight one. At a corner, attach b on both missing legs.

`What is a matrix product state, and how do we compress it?`

A matrix product state, abbreviated MPS, represents a many-index array as a chain of small tensors. Despite the name, it need not represent a quantum state. Here it represents the weights left after contracting some rows:

$$B(d_1, \ldots, d_L) = \sum_{\alpha_1, \ldots, \alpha_{L-1}} B^{[1]}_{1 d_1 \alpha_1} B^{[2]}_{\alpha_1 d_2 \alpha_2} \ldots B^{[L]}_{\alpha_{L-1} d_L 1}. \tag{19}$$

The d_i are open lattice indices. The α_i connect adjacent tensors in the chain. Each internal index α_i retains at most χ values. At fixed χ, the chain stores at most $2L\chi^2$ entries, growing linearly with the lattice width. With large enough bonds, the representation is exact. Truncating the bonds can introduce an approximation.

SVD of an Ising boundary. Group the boundary legs to the left and right of one MPS bond into indices x and y. This views the boundary as a matrix:

$$B_{xy} \approx \sum_{\alpha=1}^{\chi} U_{x\alpha} s_\alpha V_{y\alpha}. \tag{20}$$

Each bond value α pairs one left-boundary pattern with one right-boundary pattern. Compression works when a few such pairs carry most of the squared singular weight. Ising entries share local interactions, unlike the independent Gaussian entries. For one absorbed row, only one horizontal bond crosses the cut. Its two-term split $F = WW^T$ leaves at most two products after summing each half-row. The boundary matrix therefore has rank at most two; at $K = 0$ it has rank one. More rows can increase the rank, so local interactions alone do not guarantee a small discarded tail. Near the transition, correlations over longer distances can require larger χ. The later rank comparison checks the resulting heat peaks.

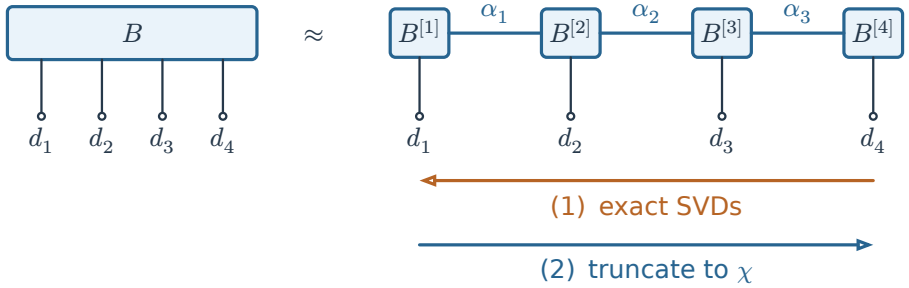


Figure 11: A four-index boundary array and its matrix product state, with the same open indices $d_1, \ldots, d_4$. Compression first sweeps right to left with exact SVDs (orange), then left to right, truncating each bond α_i to at most χ (blue).

Compression sweeps. An MPS has the gauge freedom introduced in Part 1 on each internal bond. Each small tensor is a *core*; its outward lattice leg is the *physical index*. The **environment** is the rest of the chain outside the core being split. Fixing the connecting index selects one vector of entries from that environment. A *canonical form* makes these vectors orthonormal, so local SVD errors measure errors in the whole boundary array. The right-to-left sweep prepares this basis without truncation. The left-to-right sweep then truncates each bond to at most χ directions. Its updated cores are *left-isometric*: grouping their left and physical indices gives a matrix with orthonormal columns.

Algorithm 3. Compress(B, χ): canonicalize and truncate an MPS

Input: Cores $B^{[i]}_{\alpha p\beta}$ for $i = 1, \ldots, L$, physical index $p = 1, 2$, cap $\chi \geq 1$.
Output: Compressed cores B, with the first $L - 1$ cores left-isometric.

▷ Make the right environment orthonormal without discarding information.
for $i = L, L-1, \ldots, 2$ **do**
  $(U, s, V^T) \leftarrow \mathrm{SVD}\left(B^{[i]}_{\alpha(p\beta)}\right)$
  $B^{[i]}_{qp\beta} \leftarrow (V^T)_{q(p\beta)}$
  $B^{[i-1]}_{\alpha pq} \leftarrow \sum_\gamma B^{[i-1]}_{\alpha p\gamma} U_{\gamma q} s_q$
end
▷ Now truncate each bond and pass its retained weights to the right.
for $i = 1, \ldots, L-1$ **do**
  $(U, s, V^T) \leftarrow \mathrm{TruncatedSVD}\left(B^{[i]}_{(\alpha p)\beta}, \chi\right)$
  $B^{[i]}_{\alpha pq} \leftarrow U_{(\alpha p)q}$
  $B^{[i+1]}_{qp\beta} \leftarrow \sum_\gamma s_q (V^T)_{q\gamma} B^{[i+1]}_{\gamma p\beta}$
end
return B.

After compression, the first $L - 1$ cores are left-isometric. Therefore $\|B\|_F = \left\|B^{[L]}\right\|_F$; the last core carries the boundary's scale.

Row tensors. For interior sites, put the site tensor in left, up, down, right order:

$$T^{[i]}_{ludr} = A_{ruld}. \tag{21}$$

Close the two outward legs at the ends of the row:

$$T^{[1]}_{1udr} = \sum_l b_l A_{ruld}, \quad T^{[L]}_{lud1} = \sum_r A_{ruld} b_r. \tag{22}$$

These outward axes now have size one.

What is the boundary MPS algorithm for contracting this network efficiently?

A boundary MPS uses this chain to summarize the rows already absorbed. The complete algorithm is:

Algorithm 4. Boundary-MPS contraction of an open square

Input: Row tensors $T^{[1]}, \ldots, T^{[L]}$ with side edges closed, edge vector b, cap χ.
Output: $g = \frac{\log Z}{L^2}$ for the open square.

$B^{[i]} \leftarrow \text{reshape}(b, (1, 2, 1))$ for $i = 1, \ldots, L$; top edge.
$\ell \leftarrow 0$; accumulated log scale.
for each row $y = 1, \ldots, L$ **do**
  ▷ Add the row, then compress the enlarged horizontal bonds.
  $C^{[i]}_{(\alpha l)d(\beta r)} \leftarrow \sum_u B^{[i]}_{\alpha u \beta} T^{[i]}_{ludr}$ for $i = 1, \ldots, L$
  $B \leftarrow \text{Compress}(C, \chi)$
  ▷ Scale the whole boundary once by changing only its last core.
  $c \leftarrow \left\| B^{[L]} \right\|_F$
  $B^{[L]} \leftarrow \frac{B^{[L]}}{c}$
  $\ell \leftarrow \ell + \log c$
end
$v \leftarrow [1]$
for $i = 1, \ldots, L$; close the bottom edge **do**
  $v_\beta \leftarrow \sum_{\alpha, p} v_\alpha B^{[i]}_{\alpha p \beta} b_p$
end
return $g = \frac{\ell + \log v_1}{L^2}$.

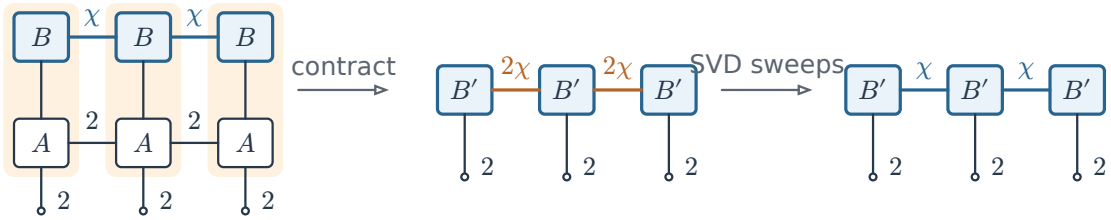


Figure 12: One row of the boundary-MPS contraction. Each shaded column contracts into one core, so the horizontal bonds grow from χ to 2χ; the SVD sweeps return them to χ. Each downward leg has dimension two.

Row growth and compression. Contract each boundary leg with the corresponding tensor in the next row. Each horizontal bond combines an MPS index with a lattice index. Its size grows from at most χ to at most 2χ. The compression sweeps bring it back to at most χ.

Edge closure and normalization. The initial cores supply the top edge; the final vector sweep supplies the bottom edge. Normalize only the last MPS core, so the represented boundary is divided by c once. The accumulated logarithms restore that scale. MPS entries may have either sign; individual entries are not probabilities.

KEY CONCEPTS
- The site tensor factors the same Ising Boltzmann weights as the spin model.
- A boundary MPS keeps correlations along the boundary in its internal bonds.
- Canonicalization makes local SVD errors meaningful; scales restore $\log Z$.

How can peaks from finite magnets locate the transition of an infinite magnet?

Compute the specific heat from neighbouring temperatures:

$$c_V(K; h) \approx K^2 \frac{g(K + h) - 2g(K) + g(K - h)}{h^2}. \tag{23}$$

The step h controls how finely we resolve the curvature. At a peak, fit a parabola through the maximum and its two neighbours in K. Its vertex gives K_{peak}; convert to $T_{\text{peak}} = \frac{1}{K_{\text{peak}}}$.

Peak drift with size. An edge interrupts a spin's neighbours. As the square grows, the fraction of spins near an edge decreases. Try $T_{\text{peak}}(L) = T_c + \frac{a}{L}$: on a plot against $\frac{1}{L}$, the intercept estimates T_c. This is a finite-range extrapolation,

not an exact formula for every size. Compare the fit using all three sizes with the fit omitting the smallest. The peak's height and motion are the physical picture; the intercept is our estimate.

③ DO

Hands-on: row contraction. Save this file in your `week10/` directory.

thermo.design.toml

```
usage = "thermo --method boundary --l 64 --k 0.45 --chi 16 --out runs/boundary64"
description = """Compute the zero-field square Ising partition function;
no unstated defaults"""

[args]
l = "Side length of the square, at least 2"
k = "Inverse temperature K=1/T, with J=k_B=1 and K>=0"
chi = "Maximum retained bond dimension, at least 1"
out = "Directory for this run's raw evidence"

[args.boundary]
method = "boundary: absorb rows with an MPS on an open square"

[writes]
evidence = """
Save the model, geometry, settings, log Z, g=log(Z)/L^2 and solver time.
Write the final tensors, normalization scales and truncation diagnostics.
Keep full floating-point precision in the saved evidence.
Record enough information to recompute the final contraction independently.
"""
```

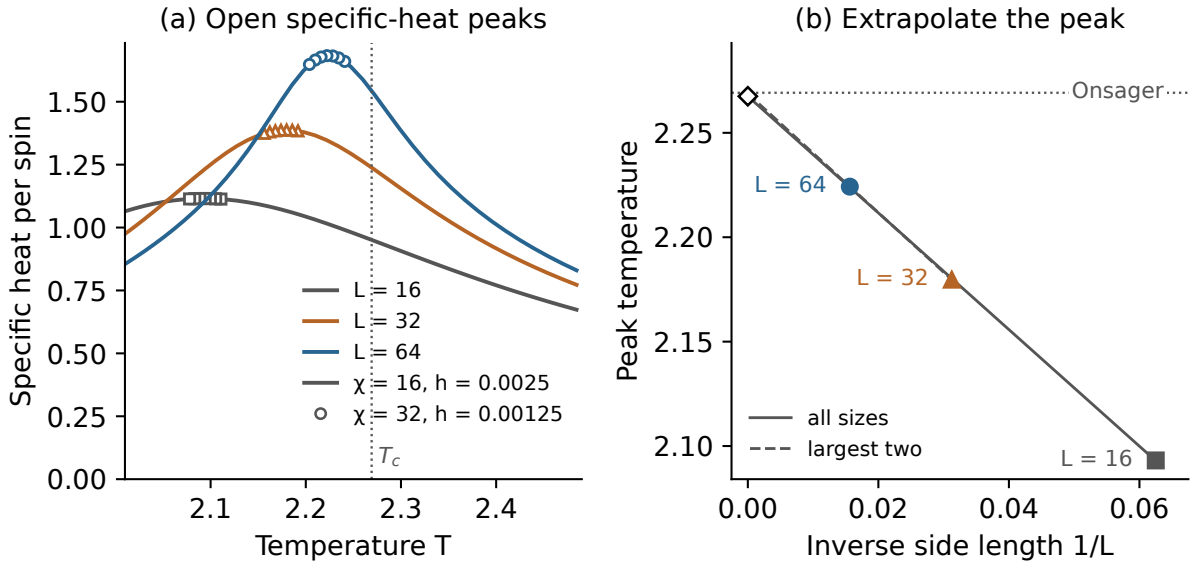
```
Build the Rust `thermo` tool described in `thermo.design.toml`. Use H=-sum_edges
s_i*s_j with s_i=±1, no field and open edges. Implement boundary MPS row absorption
with canonical SVD compression. Use W=[[sqrt(cosh K),sqrt(sinh K)],[sqrt(cosh K),-
sqrt(sinh K)]], A[r,u,l,d]=sum_s W[s,r]W[s,u]W[s,l]W[s,d], and b=[1/sqrt(cosh K),0] on
every missing bond.
```

Install the tool from its crate directory, then run the stated case:

```
cargo install --path .
thermo --method boundary --l 64 --k 0.45 --chi 16 --out runs/boundary64
```

```
Use a script to sweep open L=16,32,64 at χ=16 over K=0.40 through 0.50, step 0.0025.
From the saved g values, compute c_V and locate each peak. Around the nearest grid
point to each peak, sample nine points spaced 0.00125 at χ=16 and 32. Fit each peak
with a three-point parabola in K. Convert its vertex to T_peak=1/K_peak. Fit
T_peak(L)=T_c+a/L at χ=32. Save `artifacts/boundary-critical.png`: c_V versus T and
the peak extrapolation. Keep the g(K) tables, timings and the full runs supporting the
L=64 peak fit.
```

Open-square specific-heat peaks. The open-square peaks move from about 2.093 to 2.180 to 2.224 as L grows from 16 to 32 to 64. They also become taller: larger regions can fluctuate together. The three-size fit gives $T_c \approx 2.268$; dropping the smallest size gives about 2.269. The peaks have not reached the transition temperature individually, but their drift reveals it.




```
Compare the peak positions after increasing χ and halving the grid spacing. At χ=32,
use every other refined sample to repeat the wider-step estimate. At χ=32, fit
L=16,32,64 and L=32,64 separately. Compare both T_c estimates with 2/log(1+sqrt(2)),
aiming for 1%. Peak positions should also agree within 1% under the rank and step
changes. On an open 4×4 square at χ=8, check g against spin enumeration at
K=0,0.44,0.7 within 1e-8. At K=0.44, check c_V with h=0.0025 against K² Var(H)/16
within 1e-3.
```

The critical temperature and energy variance. Onsager's value is $T_c \approx 2.269185$. The reference estimate differs by about 0.074%. The small-square energy-variance calculation gives $c_V \approx 0.439397$. Keep the peak-refinement results and small checks in a table beside your figure.

Part 3: Coarse-grain the network with TRG

Goal: follow the same transition on periodic squares up to 1024×1024 and estimate T_c from their specific-heat peaks.

② IMAGINE

```
What is real-space renormalization group, and why is it useful?
```

Near the Ising transition, spins fluctuate together over many lattice spacings. We want to understand how these larger regions interact. A real-space renormalization group step sums over local details and describes what remains using coarser variables. New effective weights describe the interactions between the larger regions. Repeated coarse-graining reveals which microscopic details still matter at long distances. This makes renormalization useful for understanding phase transitions. Here we use that idea to follow the transition in a much larger magnet. A 1024×1024 square has over a million spins, yet needs only 20 TRG rounds.

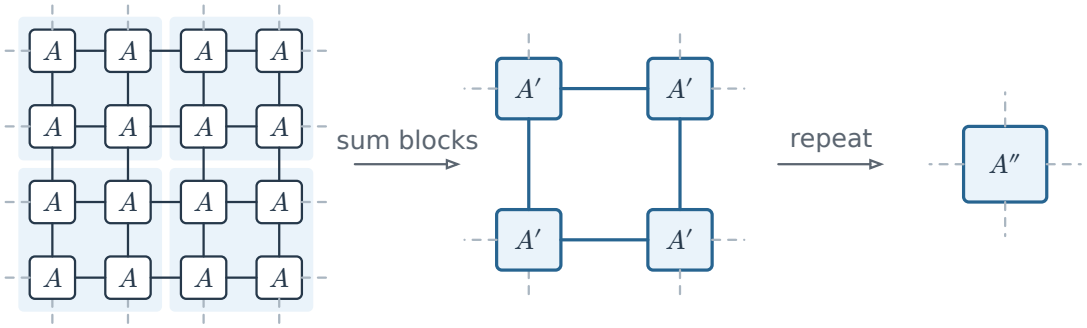


Figure 13: Schematic real-space blocking, not the TRG update. Summing within each shaded block replaces 16 microscopic weights by 4 effective ones; repeating the step reaches still larger scales. Dashed stubs show that the lattice continues.

What is the tensor renormalization group algorithm for contracting the Ising network?

The tensor renormalization group, abbreviated TRG, splits site tensors by SVD and regroups their factors. The split can discard directions; the regrouping is an exact contraction. For a periodic square with power-of-two side L, use $n = 2\log_2 L$ steps.

Algorithm 5. Square-lattice TRG

Input: $A_{ruld}(K)$ with equal leg sizes, power-of-two side $L \geq 2$, cap $\chi \geq 1$.
Output: $g = \frac{\log Z}{L^2}$ for the periodic square.

$(n, N, \ell) \leftarrow (2\log_2 L, L^2, 0)$
for $j = 1, \ldots, n$ **do**
  ▷ Each of the N remaining tensors contributes the same scale.
  $c_j \leftarrow \max|A|$
  $A \leftarrow \frac{A}{c_j}$
  $\ell \leftarrow \ell + N \log c_j$
  ▷ Alternating sites use the two diagonal cuts.
  $(S_1(l,u,q), S_3(q,r,d)) \leftarrow \mathrm{Split}(A, (l,u), (r,d), \chi)$
  $(S_2(d,l,q), S_4(q,u,r)) \leftarrow \mathrm{Split}(A, (d,l), (u,r), \chi)$
  ▷ Four half-tensors form one new site, leaving $\frac{N}{2}$ sites.
  $A'_{ruld} \leftarrow \sum_{w,a,b,\gamma} S_1(w,a,r) S_2(a,b,u) S_3(l,b,\gamma) S_4(d,\gamma,w)$
  $(A, N) \leftarrow (A', \frac{N}{2})$
end
▷ Close the last tensor by joining opposite legs.
$z \leftarrow \sum_{i,j} A_{ijij}$
return $g = \frac{\ell + \log z}{L^2}$.

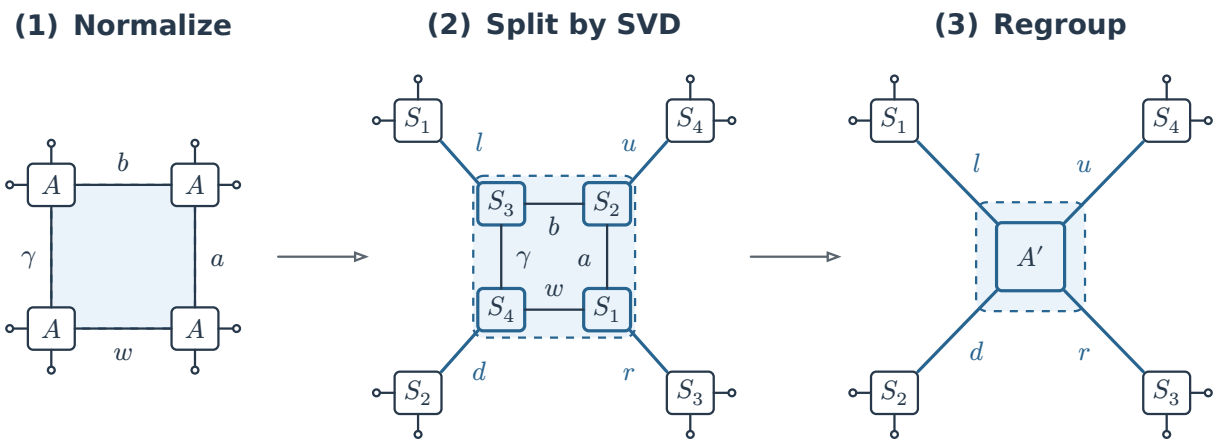


Figure 14: One TRG round around the shaded plaquette. (1) Divide each A by c and record $N \log c$. (2) Split each site along a diagonal, keeping at most χ directions. (3) Sum the ring indices w, a, b, γ to form A'_{ruld}. The white outer halves join adjacent plaquettes; the new lattice bonds follow the diagonals.

Split axes and ring indices. The split calls use the old site's (r, u, l, d) axes. Their new index q labels retained singular directions. Use the two cuts on alternating sites of a checkerboard. For the shaded square, called a *plaquette*, rename the factor indices as follows:

Ring position	Split output axes	Indices in the sum
Lower right	$S_1(l,u,q)$	$S_1(w,a,r)$
Upper right	$S_2(d,l,q)$	$S_2(a,b,u)$
Upper left	$S_3(q,r,d)$	$S_3(l,b,\gamma)$
Lower left	$S_4(q,u,r)$	$S_4(d,\gamma,w)$

Read each tuple in the listed axis order, as in the regrouping assignment. Each old index w, a, b, γ labels one inner wire and occurs in two factors. Sum each over its current dimension; the four retained wires become the new axes (r, u, l, d) of A'. These new direction names follow the rotated lattice in the right-hand panel. The outer halves remain connected to adjacent plaquettes.

Lattice size. Splitting N four-leg tensors creates $2N$ three-leg factors. Grouping four factors into one tensor leaves $\frac{N}{2}$ tensors. The coarse lattice is rotated relative to the original square. After two steps, the side length is halved. Choose an even number n of steps, so the original square has $L = 2^{\frac{n}{2}}$ and $N = 2^n$ sites. Periodic boundaries let every site use the same tensor, including across the edges.

Normalization and periodic closure. Before step j, there are 2^{n-j+1} identical site tensors. Dividing each by c_j removes that many factors of c_j from Z. Their contribution to $\log Z$ is therefore $2^{n-j+1} \log c_j$. After the final step, join opposite legs in pairs:

$$z = \sum_{i,j} A_{ijij}. \tag{24}$$

The two sums are independent. Setting all four indices equal would define a different network.

Reading the original method. Levin and Nave's original TRG paper, Figure 3, shows square-lattice splitting and regrouping. Figure 2 introduces the corresponding honeycomb construction. The paper's Ising example uses a triangular lattice; do not copy its critical coupling into the square-lattice calculation.

```
Why is the transition a problem of many length scales?
```

Away from the transition, energy fluctuations have a characteristic spatial scale. Near the transition, fluctuations extend over many scales. Increasing L gives them more room to grow. Increasing χ lets the tensors retain more of their correlations. We will follow the specific-heat peak while changing these two controls separately.

KEY CONCEPTS
- TRG alternates two local SVD splits and an exact regrouping.
- Two steps halve the square's side length.
- Scale factors carry the tensor count; periodic closure pairs opposite legs.

3 DO

Hands-on: coarse-graining. Append this block to `thermo.design.toml`:

thermo.design.toml
```
[args.trg]
method = "trg: coarse-grain a periodic square; L must be a power of two"

[writes.trg]
evidence = """
Save each step's scale, split factors and singular values, plus the final tensor.
"""
```

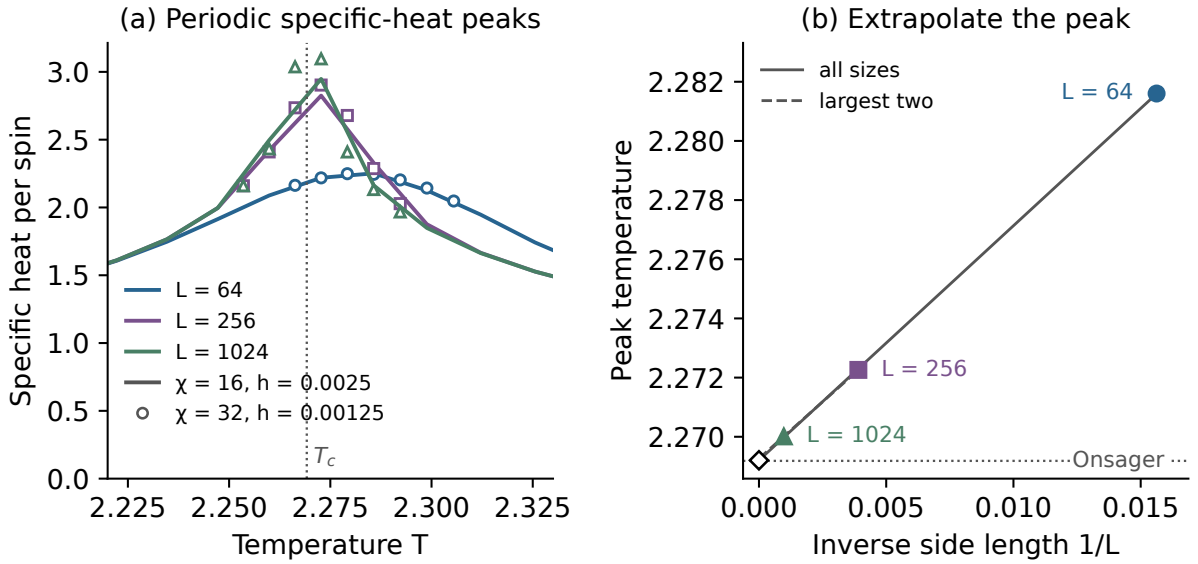
```
Add TRG to `thermo` using `thermo.design.toml`. Port the two SVD splits and regrouping
from
https://github.com/GiggleLiu/BackwardsLinalg.jl/blob/b16273a2d695fef31334085e40f9e0341
a1ec0f0/examples/trg.jl
Use n=2*log2(L) steps and max-absolute-entry normalization. Replace the sample's final
closure with sum_{i,j} A[i,j,i,j].
```

Reinstall the updated tool, then compute a million-spin torus:

```
cargo install --path . --force
thermo --method trg --l 1024 --k 0.44 --chi 16 --out runs/trg1024
```

```
Repeat the peak study with periodic TRG at L=64,256,1024. Scan K=0.40 through 0.50 in
steps of 0.0025 at χ=16. Refine nine points around each nearest peak-grid point,
spacing 0.00125, at χ=16 and 32. Repeat Part 2's peak fits, step/rank comparisons and
size fits. Save `artifacts/trg-critical.png`: c_V versus T and the periodic peak
extrapolation. Keep g(K), timings and the full runs supporting the L=1024 peak fit.
```

Periodic-square specific-heat peaks. The periodic peaks move from about 2.282 to 2.272 to 2.270 as L grows from 64 to 256 to 1024. The three-size fit gives $T_c \approx 2.2692$. Periodic boundaries remove the open square's surface. Here the peaks approach from above rather than below. Fit each geometry separately: both point to the same bulk transition.




```
Report the periodic T_c estimate and its rank and step dependence. At χ=32, compare
fits using L=64,256,1024 and L=256,1024. Aim for 1% agreement with 2/log(1+sqrt(2))
and peak stability within 1%. Check g against periodic spin enumeration at K=0.44:
L=2,χ=16 within 1e-8; L=4,χ=8 within 2e-3. Count one bond per site in each positive
direction, including the distinct parallel bonds at L=2. At L=2, check c_V with
h=0.0025 against K² Var(H)/4 within 1e-3.
```

The fit window and energy variance. Dropping $L = 64$ gives about 2.2693 in the reference. The periodic two-by-two energy-variance calculation gives $c_V \approx 0.404278$. Summarize the peak shifts and compute time in a table. The refined $\chi = 32$ runs carry the final estimate; $\chi = 16$ provides the broad view first.

KEY CONCEPTS

- The specific-heat peak grows as correlated fluctuations span larger regions.
- Boundary conditions move finite-size peaks without changing the bulk critical temperature.
- Size extrapolation and local peak refinement turn the curves into a physical estimate.

Save the evidence and prepare to show your work

Keep `svd.png`, `boundary-critical.png` and `trg-critical.png` locally in `artifacts/`. Keep the $g(K)$ tables, peak-stencil runs, small checks and peak-refinement table with them.

```
Update `README.md` with reproduction and verification commands, environment, data
sources, settings and boundary conditions. Keep generated data, tensors, scales,
figures and archives out of Git. Commit source, `thermo.design.toml`, comparison
scripts and `README.md`. Push source and documentation.
```

At the next session, show how the peak grows and moves. Explain how your two estimates of T_c follow from the curves. Connect that result to one row absorption and one TRG round.

Optional extension. Resolve the peak height by refining h and χ, then test its logarithmic growth with L.